NAME: Jeevitha.S
PROJECT TOPIC:Food Delivery Management System

Assignment 8: Basic SQL Queries and Aggregate Functions

DBMS Coursework | Data Retrieval, Filtering, Sorting & Aggregations

OBJECTIVES:

This assignment demonstrates filtering, sorting, grouping, grouped filtering and aggregate functions using the

SQL dialect and naming note: The scripts use MySQL 8.0. The project design calls its order table *Order*, but ORDER is a
SQL keyword. To keep every command executable without quoting a reserved word, this report names that table Food_Order.
It represents the same Order entity in the finalized design. All other table names and relationships are retained.

1. Filter orders using WHERE

```
SELECT Order_ID, Total_Amount, Order_Status
FROM Food_Order
WHERE Order_Status = 'Delivered'
ORDER BY Order_ID;
```

Expected output: Returns only orders whose status is Delivered.
Order_ID Total_Amount Order_Status
5001 360.00 Delivered
5002 280.00 Delivered
5006 500.00 Delivered

2. Sort orders using ORDER BY

```
SELECT Order_ID, Order_Date, Total_Amount
FROM Food_Order
ORDER BY Total_Amount DESC;
```

Expected output: Displays orders from highest to lowest total amount.
Order_ID Order_Date Total_Amount

5006 2026-10-23 18:30:00 500.00
5001 2026-10-20 12:10:00 360.00
5003 2026-10-21 19:20:00 350.00
5004 2026-10-22 12:40:00 300.00
5002 2026-10-20 13:05:00 280.00
5005 2026-10-22 20:15:00 240.00

3. Group orders by status

```
SELECT Order_Status, COUNT(*) AS Number_of_Orders
FROM Food_Order
GROUP BY Order_Status
ORDER BY Order_Status;
```

Expected output: Counts the orders in each status group.
Order_Status Number_of_Orders
Cancelled 1
Delivered 3
Out for Delivery 1
Preparing 1

4. Filter groups using HAVING

```
SELECT Restaurant_ID, COUNT(*) AS Number_of_Orders
FROM Food_Order
GROUP BY Restaurant_ID
HAVING COUNT(*) >= 2;
```

Food Delivery Management System | DBMS Page 2Expected output: Shows only restaurants with at least two orders.
Restaurant_ID Number_of_Orders
101 2
102 2
103 2

5. COUNT() aggregate

```
SELECT COUNT(*) AS Total_Customers
FROM Customer;
```

Expected output: Counts all customer records.
Total_Customers
5

6. SUM() aggregate

SELECT SUM(Total_Amount) AS Total_Order_Value
FROM Food_Order;
Expected output: Adds the total amounts of all six orders, including the cancelled sample order.
Total_Order_Value
2030.00

7. AVG() aggregate

SELECT ROUND(AVG(Rating),2) AS Average_Restaurant_Rating
FROM Restaurant;
Expected output: Calculates the mean rating of the three restaurants.
Average_Restaurant_Rating
4.23

8. MIN() and MAX() aggregates

SELECT MIN(Total_Amount) AS Minimum_Order,
MAX(Total_Amount) AS Maximum_Order
FROM Food_Order;
Expected output: Finds the smallest and largest order totals.
Minimum_Order Maximum_Order
240.00 500.00

>> SQL Queries & Results Demonstration

Query 1: Filtering Records using WHERE WHERE Clause

```
SELECT Item_ID, Item_name, Category, Price
FROM Menu_Item
WHERE Price > 250.00 AND Availability = TRUE;
```

Explanation: Filters active menu items priced above ₹250.00 to identify premium dish offerings available for ordering.

Item_ID	Item_name	Category	Price (₹)
201	All American Burger	Main Course	320.00
203	Special Chicken Biryani	Main Course	380.00
204	Paneer Butter Masala	Main Course	290.00
205	Ghee Rice & Chicken Kebab	Combo	310.00
207	Sourdough Pepperoni Pizza	Main Course	550.00

Query 2: Sorting Records using ORDER BY ORDER BY Clause

```
ORDER BY Rating DESC, Restaurant_name ASC;
```

Explanation: Sorts restaurants primarily by highest customer rating in descending order, with name as tie-breaker.

Restaurant_ID	Restaurant_name	Phone	Rating
2	Meghana Foods	08025535678	4.8
1	Truffles Gourmet	08025531234	4.7
4	Pizza Bakery	08025538888	4.5
3	Empire Restaurant	08025539999	4.3

Query 3: Aggregate Functions (COUNT, SUM, AVG, MIN, MAX) Aggregates

```
FROM Order_Table;
```

Explanation: Computes key platform transaction KPIs including volume, total revenue, average, minimum, and peak order amounts.

Total_Orders	Revenue (₹)	Avg_Order_Value (₹)	Min_Order_Val (₹)	Max_Order_Val (₹)
5	2610.00	522.00	370.00	730.00

>> SQL Queries & Results Demonstration (Continued)

Query 4: Grouping Records using GROUP BY GROUP BY Clause

```
GROUP BY Order_status;
```

Explanation: Aggregates order volume and total revenue grouped by current operational status (Delivered, Preparing, Pending).

Order_status	Order_Count	Status_Revenue (₹)
Delivered	3	1510.00
Preparing	1	370.00
Pending	1	730.00

Query 5: Filtering Grouped Results using HAVING HAVING Clause

```
HAVING SUM(Total_amount) > 500.00;
```

Explanation: Groups orders by restaurant ID and applies `HAVING` to filter and retain only high-performing restaurants with revenue > ₹500.

Restaurant_ID	Total_Orders	Sales (₹)
2	2	1050.00
4	1	730.00

Query 6: Combined Multi-Clause SQL Query Complex Query

```
SELECT Category, COUNT(Item_ID) AS Item_Count, AVG(Price) AS Avg_Category_Price
FROM Menu_Item
WHERE Availability = TRUE
GROUP BY Category
HAVING AVG(Price) > 150.00
ORDER BY Avg_Category_Price DESC;
```

Explanation: Integrates `WHERE` (available items), `GROUP BY` (category), `HAVING` (avg price > ₹150), and `ORDER BY` (descending price).

Category	Item_Count	Avg_Category_Price (₹)
Main Course	4	385.00
Combo	1	310.00

>> Summary of SQL Query Operations

The queries demonstrate effective usage of SQL DML constructs: `WHERE` operates at the individual row level before grouping; `GROUP BY` partitions rows into logical subsets; `HAVING` filters grouped rows using aggregate conditions; and `ORDER BY` controls presentation sequence. All queries executed with zero errors on the populated schema.